Algorithms Analysis

By

Aman Ullah

Lecture No 7

(Quick Sort)

Today's Topics

In this lecture we will cover the following

- Quick Sort

Quick Sort

- Quick Sort is a partition based algorithm.
- It was designed by C. A. R. Hoere in 1960's.
- It is the most frequently used algorithm and is found in most sorting libraries (C and Java libraries).
- The quicksort algorithm has a worst-case running time of $O(n^2)$ on an input array of n numbers.
- Its expected running time is $\Theta(n \lg n)$.
- It also has the advantage of sorting in place.
- It is not stable sort but it can be made stable.
- Quicksort, like merge sort, applies the divide-and-conquer paradigm.
- The three-step divide-and-conquer process for sorting a typical subarray A[p . . r] is as follows:

Quick Sort (Contd…)

- **Divide:** Partition (rearrange) the array A[p . . r] into two (possibly empty) subarrays A[p . . q – 1] and A[q + 1. . r] such that each element of A[p . . q – 1] is less than or equal to A[q], which is, in turn, less than or equal to each element of A[q + 1. . r]. Compute the index q as part of this partitioning procedure.
- **Conquer:** Sort the two subarrays A[p . . q – 1] and A[q + 1. . r] by recursive calls to quicksort.
- **Combine:** Because the subarrays are already sorted, no work is needed to combine them (like Merge Sort): the entire array A[p . . r] is now sorted.
- Following is the Quick Sort algorithm.

Quick Sort Algorithm

```
QUICKSORT(A, p, r)
1  if p < r
2      q = PARTITION(A, p, r)
3      QUICKSORT(A, p, q − 1)
4      QUICKSORT(A, q + 1, r)
```

Here p denotes the first index of array A and r denotes the last index.
q will contain the index of the pivot element.
Left subarray

Right subarray

To sort an entire array A, the initial call is QUICKSORT$(A, 1, A.length)$.

- The PARTITION procedure partitions the array A[p . . r] into three subarrays about a pivot element ‘x’.
  - A[p . . q – 1] whose elements are less than or equal to ‘x’.
  - A[q] = x (i.e. pivot or partitioning element)
  - A[q + 1 . . r] whose elements are greater than ‘x’.

- The key to the algorithm is the PARTITION procedure, which rearranges the subarray A[p . . r] in place.

Quick Sort Algorithm (Contd…)

```
PARTITION(A, p, r)
1  x = A[r]
2  i = p − 1
3  for j = p to r − 1
4      if A[j] ≤ x
5          i = i + 1
6          exchange A[i] with A[j]
7  exchange A[i + 1] with A[r]
8  return i + 1
```

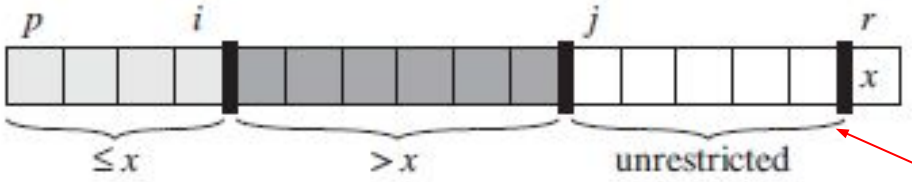
Run this method on the following array 'A'

1	2	3	4	5	6	7	8
5	3	8	6	2	7	9	4

Here p = 1 and r = 8. We partition it w.r.t. last element i.e. 4 which is stored in variable 'x'

- PARTITION procedure always selects an element x = A[r] as a ***pivot*** element around which to partition the subarray A[p . . r].
- As the procedure runs, it partitions the array into four (possibly empty) regions.
- At the start of each iteration of the **for** loop in lines 3–6, the regions satisfy certain properties, shown in following Figure.

Quick Sort Algorithm (Contd…)



Elements yet to be processed

Figure 7.2 The four regions maintained by the procedure PARTITION on a subarray $A[p..r]$. The values in $A[p..i]$ are all less than or equal to x, the values in $A[i+1..j-1]$ are all greater than x, and $A[r] = x$. The subarray $A[j..r-1]$ can take on any values.

- At the beginning of each iteration of the loop of lines 3–6, for any array index k,

1. If p ≤ k ≤ i, then A[k] ≤ x.
2. If i+1 ≤ k ≤ j – 1, then A[k] > x.
3. If k = r, then A[k] = x.

(Pivot element will get its correct position)

- The indices between j and r – 1 are not covered by any of the three cases, and the values in these entries have no particular relationship to the pivot x.

Quick Sort Algorithm (Contd…)

- Following Figure shows how PARTITION works on an 8-element array.

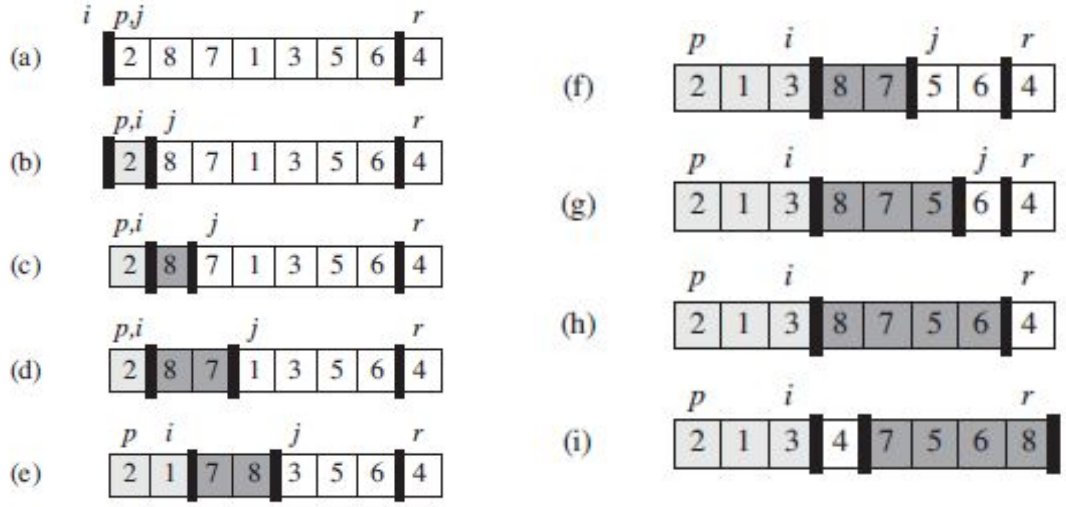


Figure 7.1 The operation of PARTITION on a sample array. Array entry $A[r]$ becomes the pivot element x. Lightly shaded array elements are all in the first partition with values no greater than x. Heavily shaded elements are in the second partition with values greater than x. The unshaded elements have not yet been put in one of the first two partitions, and the final white element is the pivot x. (a) The initial array and variable settings. None of the elements have been placed in either of the first two partitions. (b) The value 2 is "swapped with itself" and put in the partition of smaller values. (c)–(d) The values 8 and 7 are added to the partition of larger values. (e) The values 1 and 8 are swapped, and the smaller partition grows. (f) The values 3 and 7 are swapped, and the smaller partition grows. (g)–(h) The larger partition grows to include 5 and 6, and the loop terminates. (i) In lines 7–8, the pivot element is swapped so that it lies between the two partitions.

Quick Sort Algorithm (Contd…)

- The final two lines of PARTITION finish up by swapping the pivot element with the leftmost element greater than x, thereby moving the pivot into its correct place in the partitioned array, and then returning the pivot's new index.
- In fact, it satisfies a slightly stronger condition: after line 2 of QUICKSORT, A[q] is strictly less than every element of A[q+1. . r].
- The running time of PARTITION on the subarray A[p . . r] is Θ (n), where n = r – p + 1

Performance of Quick Sort Algorithm

- The running time of quicksort depends on whether the partitioning is balanced or unbalanced, which in turn depends on which elements are used for partitioning.
- If the partitioning is balanced, the algorithm runs asymptotically as fast as merge sort (i.e. O(n log n)), as it will always make almost two equal halves.
- If the partitioning is unbalanced, however, it can run asymptotically as slowly as insertion sort (i.e. $O(n^2)$).
- We shall informally investigate how quicksort performs under the assumptions of balanced versus unbalanced partitioning.

Worst Case Partitioning

- The worst-case behavior for quicksort occurs when the partitioning routine produces one subproblem with n – 1 elements and one with 0 elements (i.e. when we choose largest or smallest element as the pivot element).
- Let us assume that this unbalanced partitioning arises in each recursive call.
- The partitioning costs $\Theta(n)$ time. Since the recursive call on an array of size 0 just returns, $T(0) = 0$ and the recurrence for the running time is

$$T(n) = T(n-1)+T(0) + \Theta(n)$$

$$= T(n-1) + \Theta(n) = \Theta(n^2)$$ (By Substitution method)

The summation

$$\sum_{k=1}^{n} k = 1 + 2 + \cdots + n ,$$

is an ***arithmetic series*** and has the value

$$\sum_{k=1}^{n} k = \frac{1}{2} n(n+1)$$

$$= \Theta(n^2) .$$

Worst Case Partitioning (Contd...)

- Thus, if the partitioning is maximally unbalanced at every recursive level of the algorithm, the running time is $\Theta(n^2)$.
- Therefore the worst-case running time of quicksort is no better than that of insertion sort.

  - T(n) = T(n-1)+cn from this recurrence, we have
  - T(n-1) = T(n-2) + c(n-1)
  - T(n-2) = T(n-3) + c(n-2)
  - T(n-3) = T(n-4) + c(n-3)
  - ……..
  - ……. so on. Now replace these values back in actual recurrence, we get
  - T(n) = cn + c(n-1) + c(n-2) + c(n-3) + … + c.3 + c.2 + c.1
  - T(n) = c(1+ 2+ 3+ …. + (n-3)+ (n-2)+ (n-1)+ n)
  - $T(n) = c(n(n+1)/2) = cn^2 + cn = \Theta(n^2)$

Best Case Partitioning

- In the most even possible split, PARTITION produces two subproblems, each of size no more than n/2, since one is of size $\lfloor n/2 \rfloor$ and one of size $\lceil n/2 \rceil - 1$ (Excluding pivot)
- In this case, quicksort runs much faster.
- The recurrence for the running time is then

  $T(n) = 2T(n/2) + \Theta(n)$; Here $\Theta(n)$ is running time of partitioning

  where we tolerate the sloppiness from ignoring the floor and ceiling and from subtracting 1.
- By case 2 of the master theorem, this recurrence has the solution $T(n) = \Theta(n \lg n)$.
- So, by equally balancing the two sides of the partition at every level of the recursion, we get an asymptotically faster algorithm.

Balanced Partitioning

- The average-case running time of quicksort is much closer to the best case than to the worst case.
- Suppose, for example, that the partitioning algorithm always produces a 9-to-1 proportional split, which at first blush seems quite unbalanced.
- We then obtain the recurrence

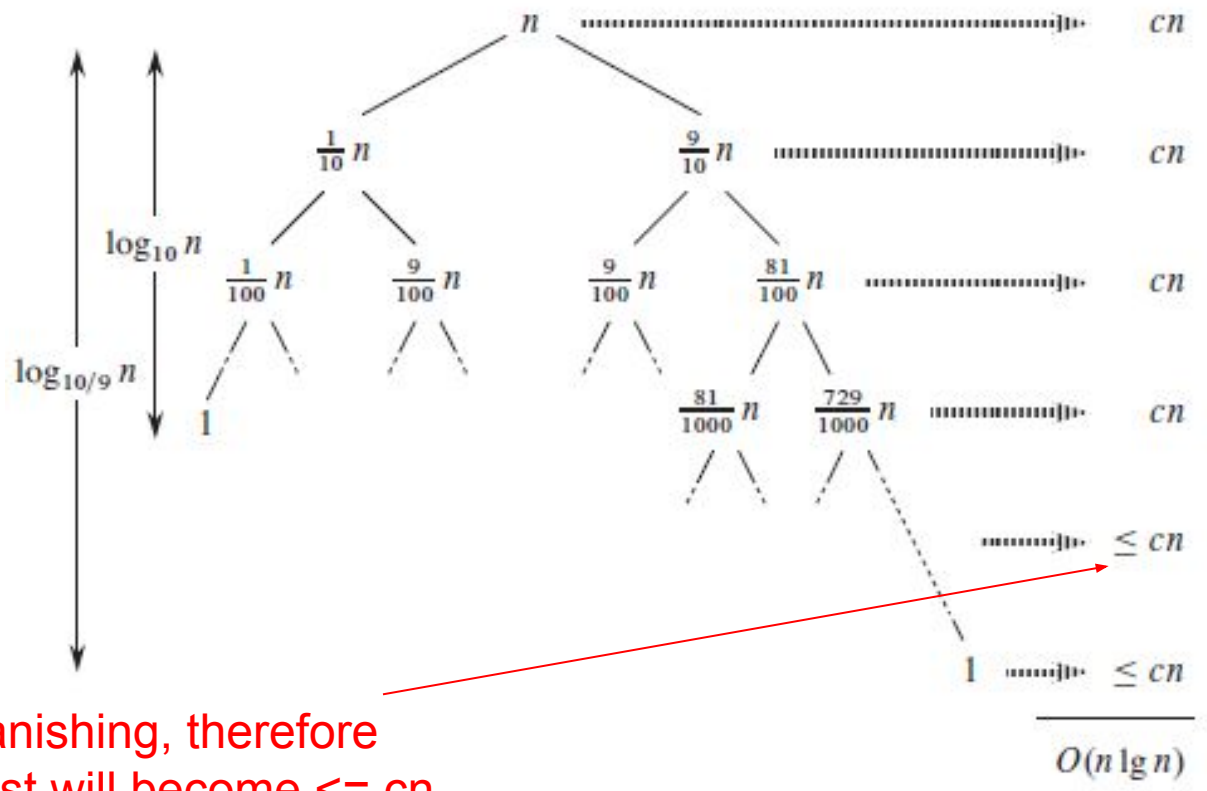
  $T(n) = T(9n/10) + T(n/10) + cn$;

  on the running time of quicksort, where we have explicitly included the constant c hidden in the $\Theta(n)$ term.

Balanced Partitioning (Contd...)

- The following figure represents the recursion tree for the above recurrence.

- This is a binary tree so Levels of tree = lg n + 1
- Cost of each level = cn So
- T(n) = cn (lg n +1)
- T(n) = cn lg n + cn
- T(n) = O(n lg n)



Left subtrees are vanishing, therefore after some level, cost will become <= cn

Balanced Partitioning (Contd…)

- Notice that every level of the tree has cost cn, until the recursion reaches a boundary condition at depth $\log_{10} n = \Theta(\lg n)$, and then the levels have cost at most cn.
- The recursion terminates at depth $\log_{10/9} n = \Theta(\lg n)$. The total cost of quicksort is therefore $O(n \lg n)$. Thus, with a 9-to-1 proportional split at every level of recursion, which intuitively seems quite unbalanced, quicksort runs in $O(n \lg n)$ time.

Randomized Version of Quick Sort

- Instead of always using A[r] as the pivot, we will select a randomly chosen element from the subarray A[p . . r].
- We do so by first exchanging element A[r] with an element chosen at random from A[p . . r].
- By randomly sampling the range p, . . . , r, we ensure that the pivot element x = A[r] is equally likely to be any of the r – p + 1 (i.e. n) elements in the subarray.
- Because we randomly choose the pivot element, we expect the split of the input array to be reasonably well balanced on average.

Randomized Version of Quick Sort (Contd…)

- The changes to PARTITION and QUICKSORT are small. In the new partition procedure, we simply implement the swap before actually partitioning.

```
RANDOMIZED-PARTITION(A, p, r)
1  i = RANDOM(p, r)
2  exchange A[r] with A[i]
3  return PARTITION(A, p, r)
```

- The new quicksort calls RANDOMIZED-PARTITION in place of PARTITION:

```
RANDOMIZED-QUICKSORT(A, p, r)
1  if p < r
2       q = RANDOMIZED-PARTITION(A, p, r)
3       RANDOMIZED-QUICKSORT(A, p, q − 1)
4       RANDOMIZED-QUICKSORT(A, q + 1, r)
```

Randomized Version of Quick Sort (Anslysis)

Worst Case Running Time:

- Let T(n) be the worst-case time for the procedure QUICKSORT on an input of size n. Then we have the recurrence

For left Subarray of pivot — For right Subarray of pivot — Partitioning cost

$$T(n) = T(q - 1) + T(n - q) + \Theta(n)$$

1	2	3	4	q	6	7	8
5	**3**	**8**	**6**	**2**	**7**	**9**	**4**

Here n=8 and q =5 (i.e. pivot)
So q-1 = 5-1 = 4 (Left subarray)
and n-q = 8-5=3 (Right subarray)

- To get worst case running time of the algorithm, we shall find the maximum value of T(n). So, we start with the following recurrence relation

Randomized Version of Quick Sort (Anslysis)

$$T(n) = \begin{cases} 1 & \text{if } n \le 1 \\ \max_{1 \le q \le n} (T(q-1) + T(n-q) + n) & \text{Otherwise} \end{cases}$$

- The worst case happens at q = 1 or q = n (i.e. if we choose first or last element as pivot, which may be largest or smallest elements).
- So, if we put q = 1, the recurrence

  $T(n) = T(q-1) + T(n-q) + n$

  becomes

  $T(n) = T(0) + T(n-1) + n$

  $= 1 + T(n-1) + n$ (since $T(n) = 1$ if $n \le 1$)

  $= T(n-1) + (n+1)$

Randomized Version of Quick Sort (Anslysis)

T(n) = T(n-2) + n + (n + 1)

= T(n-3) + (n – 1) + n + (n + 1)

= T(n-4) + (n – 2) + (n – 1) + n + (n + 1)

= T(n-k) + (n-(k-2)) + . . . + (n – 1) + n + (n + 1)

= $T(n-k)+\sum_{i=-1}^{k-2}(n-i)$

- In order to get base case i.e. T(1) = 1, we set

n – k = 1 or k = n-1 and get

T(n) = $T(n-(n-1))+\sum_{i=-1}^{(n-1)-2}(n-i)$

= $T(1)+\sum_{i=-1}^{n-3}(n-i)$

Randomized Version of Quick Sort (Anslysis)

$$T(n) = 1 + (3 + 4 + 5 + \ldots + (n-1) + n + (n+1))$$

$$\leq \sum_{i=1}^{n+1} i = (n+1)(n+2)/2$$

We have used ≤ symbol here because it misses '2' in the series

$$T(n) \leq (n+1)(n+2)/2 \in \Theta(n^2)$$

- Similarly, if we choose q = n, the recurrence relation

$$T(n) = T(q-1) + T(n-q) + n$$

will again becomes

$$T(n) = T(n-1) + T(n-n) + n$$

$$= T(n-1) + T(0) + n$$

which will again yield $T(n) \in \Theta(n^2)$

- Thus worst case running time of randomized quick sort is also $\Theta(n^2)$.

Thanks

Q & A